

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И
МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена трудового Красного Знамени федеральное государственное
бюджетное
образовательное учреждение высшего образования
«Московский технический университет связи и информатики»**

Кафедра Математическая кибернетика и информационные технологии

**Лабораторная работа №1
Инструмент сборки Maven
по дисциплине
«Информационные технологии и программирование»**

Выполнил: студент гр. БВТ2403
Толибов Р.Ш.

Руководитель:

Москва, 2026 г.

Цель работы

Изучить инструмент автоматизации сборки Apache Maven, освоить управление зависимостями, настройку жизненного цикла проекта, а также интеграцию инструментов логирования, сериализации и статического анализа кода.

Задачи

- Создать базовый Maven-проект и настроить exes-maven-plugin для запуска приложения.
- Добавить зависимость для логирования (SLF4J + Log4j) и заменить System.out на логгер.
- Добавить зависимость Jackson для работы с JSON и создать класс сериализации/десериализации.
- Изучить транзитивные зависимости проекта.
- Добавить плагин SpotBugs, выполнить анализ кода, исправить найденные ошибки.
- Настроить автоматическое завершение сборки при обнаружении ошибок выше порога Medium.

Ход работы

1. Создание Maven-проекта и настройка exec-maven-plugin

В качестве основы для лабораторной работы был взят проект из прошлого семестра — реализация хеш-таблицы на Java с использованием метода цепочек (chaining). Проект содержит три класса: `HashTable<K,V>`, `Student` и `Main`.

Была создана стандартная структура Maven-проекта:

Папка / Файл	Назначение
src/main/java/org/example/	Исходные .java файлы
src/main/resources/	Ресурсы (log4j2.xml)
pom.xml	Файл конфигурации Maven

В файл `pom.xml` был добавлен плагин `exec-maven-plugin`, позволяющий указать главный класс непосредственно в конфигурации:

```
<plugin>
  <groupId>org.codehaus.mojo</groupId>
  <artifactId>exec-maven-plugin</artifactId>
  <version>3.1.0</version>
  <configuration>
    <mainClass>org.example.Main</mainClass>
  </configuration>
</plugin>
```

Команда для сборки и запуска проекта:

```
mvn clean install exec:java
```

2. Добавление логирования (SLF4J + Log4j)

В секцию `<dependencies>` файла `pom.xml` были добавлены три зависимости: `SLF4J API` как интерфейс логирования, `log4j-slf4j2-impl` как реализация, и `log4j-core` как ядро `Log4j`:

```
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
```

```

    <version>2.0.9</version>
</dependency>
<dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-slf4j2-impl</artifactId>
    <version>2.21.1</version>
</dependency>
<dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-core</artifactId>
    <version>2.21.1</version>
</dependency>

```

В классе Main все вызовы `System.out.println()` были заменены на вызовы логгера:

```

private static final Logger logger =
    LoggerFactory.getLogger(Main.class);
// вместо System.out.println("Размер таблицы: " +
students.size())
logger.info("Размер таблицы: {}", students.size())

```

3. Добавление Jackson и сериализация/десериализация JSON

В `pom.xml` добавлена зависимость `jackson-databind`:

```

<dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.16.0</version>
</dependency>

```

После добавления зависимости была выполнена команда `mvn dependency:tree` для просмотра транзитивных зависимостей. Jackson автоматически подтянул:

- `jackson-core` — низкоуровневая обработка JSON
- `jackson-annotations` — аннотации для маппинга

В класс `Student` добавлены геттеры и пустой конструктор (требуется для корректной работы Jackson). Создан класс `StudentSerializer`:

```

public class StudentSerializer {
    private static final Logger logger =
        LoggerFactory.getLogger(StudentSerializer.class);
    private final ObjectMapper mapper = new ObjectMapper();

```

```

    public String toJson(Student student) {
        try {
            return mapper.writerWithDefaultPrettyPrinter()
                .writeValueAsString(student);
        } catch (Exception e) {
            logger.error("Ошибка сериализации: {}",
e.getMessage());
            return null;
        }
    }

    public Student fromJson(String json) {
        try {
            return mapper.readValue(json, Student.class);
        } catch (Exception e) {
            logger.error("Ошибка десериализации: {}",
e.getMessage());
            return null;
        }
    }
}

```

Результат работы сериализации — студент преобразуется в JSON и успешно восстанавливается обратно:

```

17:45:17 [INFO] JSON: {
    "name" : "Иван",
    "surname" : "Иванов",
    "age" : 20,
    "gpa" : 4.5
}
17:45:17 [INFO] Восстановлен: Иван Иванов (возраст: 20, ср.
балл: 4.5)

```

4. Добавление SpotBugs и анализ кода

В секцию `<build><plugins>` файла `pom.xml` добавлен плагин SpotBugs с настройками для дополнительного задания — сборка завершается с ошибкой если найдены баги уровня Medium и выше:

```

<plugin>
  <groupId>com.github.spotbugs</groupId>
  <artifactId>spotbugs-maven-plugin</artifactId>
  <version>4.9.3.0</version>
  <configuration>

```

```
<threshold>Medium</threshold>
<effort>Max</effort>
<failOnError>true</failOnError>
</configuration>
</plugin>
```

Выполнена команда:

```
mvn spotbugs:check
```

SpotBugs обнаружил одну ошибку уровня High в классе HashTable:

Уровень	Код ошибки	Описание
High	RV_ABSOLUTE_VALUE_OF_HASHCODE	Некорректное использование Math.abs() для получения абсолютного значения hashCode(). При Integer.MIN_VALUE метод возвращает отрицательное число.

5. Исправление найденной ошибки

Ошибка находилась в методе hash() класса HashTable. Исходный код:

```
private int hash(K key) {
    return Math.abs(key.hashCode()) % capacity; // BAD
}
```

Проблема: метод Math.abs() не гарантирует положительный результат для Integer.MIN_VALUE, так как $|Integer.MIN_VALUE| = Integer.MAX_VALUE + 1$, что вызывает переполнение и возврат отрицательного значения. Это может привести к `ArrayIndexOutOfBoundsException`.

Исправленный код использует побитовое И с маской 0x7fffffff, которая обнуляет знаковый бит:

```
private int hash(K key) {
    return (key.hashCode() & 0x7fffffff) % capacity; // FIXED
}
```

После исправления повторный запуск SpotBugs показал:

```
BugInstance size is 0
Error size is 0
No errors/warnings found
BUILD SUCCESS
```

Итоговый файл pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>org.example</groupId>
  <artifactId>3lab</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-
8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-api</artifactId>
      <version>2.0.9</version>
    </dependency>
    <dependency>
      <groupId>org.apache.logging.log4j</groupId>
      <artifactId>log4j-slf4j2-impl</artifactId>
      <version>2.21.1</version>
    </dependency>
    <dependency>
      <groupId>org.apache.logging.log4j</groupId>
      <artifactId>log4j-core</artifactId>
      <version>2.21.1</version>
    </dependency>
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
      <version>2.16.0</version>
    </dependency>
  </dependencies>
```

```
<build>
  <plugins>
    <plugin>
      <groupId>org.codehaus.mojo</groupId>
      <artifactId>exec-maven-plugin</artifactId>
      <version>3.1.0</version>
      <configuration>
        <mainClass>org.example.Main</mainClass>
      </configuration>
    </plugin>
    <plugin>
      <groupId>com.github.spotbugs</groupId>
      <artifactId>spotbugs-maven-plugin</artifactId>
      <version>4.9.3.0</version>
      <configuration>
        <threshold>Medium</threshold>
        <effort>Max</effort>
        <failOnError>true</failOnError>
      </configuration>
    </plugin>
  </plugins>
</build>
</project>
```

Вывод

В ходе лабораторной работы был изучен инструмент автоматизации сборки Apache Maven. Был создан Maven-проект на основе реализации хеш-таблицы из прошлого семестра и настроен `exec-maven-plugin` для запуска приложения.

Добавлены зависимости `SLF4J` и `Log4j`, все вызовы `System.out.println()` заменены на структурированные сообщения логгера с указанием уровня `INFO/ERROR`. Добавлена библиотека `Jackson` для сериализации объектов `Student` в `JSON` и их восстановления обратно. При изучении транзитивных зависимостей через команду `mvn dependency:tree` было обнаружено, что `jackson-databind` автоматически подтягивает `jackson-core` и `jackson-annotations`.

Плагин `SpotBugs` обнаружил реальную ошибку в методе `hash()` класса `HashTable` — некорректное использование `Math.abs()` с `hashCode()`. Ошибка была исправлена с использованием побитовой маски `0x7fffffff`. Повторный анализ подтвердил отсутствие ошибок. Настройка `failOnError=true` гарантирует, что сборка завершится с ошибкой при обнаружении дефектов уровня `Medium` и выше.

<https://github.com/TRuslan666/ITaP/tree/main/лаб1>

